Agentic Development with Harness Engineering

AI-Driven Software Engineering

Humans Steer. Agents Execute.

Ted Jongseok Won

JBUG Korea | April 29, 2026

Disclaimer

This presentation was created with the help of AI.

There may be errors in the content, and the author has not been able to fully verify everything.

If you find any mistakes, please let me know.

Agenda

00	Survival Strategy for Developers in the AI Era	3 min
01	AI Assistant for Work — MCP, Agents, Skills	8 min
02	Tips for Using Claude Code for Free	5 min
03	**Agentic Development with Harness**	**22 min**
Q&A	Key Takeaways & Q&A	7 min

> This is not a technical lecture. It's a **real-world experience sharing from a fellow developer**.

How much are you using AI at work right now?

Let us know in the chat!

1	2	3
Not at all	Sometimes	Every day

> "I've been working as a software engineer for **15+ years**.
> But my workflow has **completely changed from just a year ago.**"

The Key Message

"10 years of software engineering experience" alone won't guarantee the next 10 years

IMPOSSIBLE WITH AI ALONE

Understanding business **context**

Asking the right **questions**

Final **judgment** and **decisions**

Result **verification** and **accountability**

INEFFICIENT WITH HUMANS ALONE

Processing massive amounts of information

Repetitive tasks

Leveraging cross-domain knowledge

Code exploration and comprehension

> "Expertise alone" vs "**Expertise + AI = Overwhelming competitive advantage**"

AI Doesn't Replace Experts — It Amplifies Them

IT is a field where **"good enough" doesn't cut it** — precision is everything

- AI sometimes gives **confidently wrong answers**
- It presents non-existent APIs, incorrect config values, and wrong commands **with certainty**
- You **cannot trust AI's responses 100%**
- You need **domain expertise and experience** to properly leverage AI

> **"AI can run fast, but you're the only one who knows the direction."**

Uncontrolled AI Is Dangerous

WHAT I WANT FROM AI	WHAT AI ACTUALLY DOES
Ask for details before proceeding	**Decides** on its own and proceeds
Verify **step by step**	Dumps **massive output all at once**
I'm **in control**	I'm **overwhelmed**

- Modifies **dozens of files** in a single request
- Proceeds with tasks **on its own judgment** that it should have asked about first

> **This is why Harness is needed.**
>
> A structure that **limits** AI's autonomy and **verifies** at each step is essential.

Humans Are the Bottleneck

AI agents work **fast and at scale** — the problem is that **humans have to review all the results**

- Changes AI made in **5 minutes** take humans **1 hour** to review
- 30 files modified at once — humans **cannot review every detail**
- Skip reviews → **quality collapse**, Do reviews → **productivity bottleneck**

> That's why **automated verification (Harness)** is essential.
> Instead of humans checking everything, **machines verify and humans only make judgments**.

Part I

AI Assistant for Work

MCP AI Assistant, Agents, Skills, OpenClaw

MCP AI Assistant — Unified Work Tool Hub

Inspired by Julia Evans' "Get your work recognized: write a brag document".

Connecting AI and work tools via **MCP (Model Context Protocol)**

Diverse Services, 120+ Tools

CATEGORY	SERVICES
Development	Jira, GitLab, Confluence
Communication	Gmail
Productivity	Calendar, Drive, Tasks
Knowledge Management	Obsidian

> **MCP** = A **standard interface** between AI models and external tools (Anthropic open protocol)

MCP AI Assistant — Usage Examples

AI seamlessly navigates multiple work systems from a single terminal

```
> Check my Jira tickets for today

> Show me my calendar for this week

> Find the security policy document on Confluence

> Draft a comment for ticket PROJ-1234

> Schedule a security review meeting for tomorrow at 10 AM
```

> Multiple services unified through one AI — minimizing context switching

Claude Code Skills & Agents

The leverage effect of one person harnessing the capabilities of 33 specialists

	SKILLS (33)	AGENTS (4)
Execution	Runs **inline** in the main conversation	Runs in an **independent** context window
Software Engineering Analogy	`import static Utils.*`	`ExecutorService.submit(task)`
Analogy	Perform tasks yourself using a manual	Delegate to a colleague, get results back
Examples	Security analysis, prompt refinement, debugging	Security review, code review
Speed	Fast	Slower (separate context)

SKILLS PACKAGE MANAGER

Install / manage / share Skills like npm packages

> Like having **hired a team of specialists** in each domain

OpenClaw — Personal AI Assistant

Self-Hosted AI Gateway (MAC MINI 32GB)

ITEM	DETAILS
Supported Messengers	Discord, Slack, Telegram, WhatsApp, iMessage
License	Open Source (MIT)
Core Principle	My device, my data, my rules

```
[Telegram]
Me: How do I add a Quarkus health check?
AI: Add the SmallRye Health extension...
```

> "AI in my pocket" — access AI from anywhere via messenger

Google Workspace CLI — AI Integration with Google Services

An integrated CLI tool built by Google — supporting **14 services**

```
npm install -g @googleworkspace/cli
```

INTEGRATION TARGET	USAGE EXAMPLES
Claude Code	**"Schedule a meeting for tomorrow morning"**, **"Upload this file to Drive"**
OpenClaw	**"What's my schedule today?"**, **"Send an email to the team"** via messenger

- All responses are **structured JSON** — AI interprets and executes immediately
- 40+ Agent Skills included out of the box

> GitHub: **github.com/googleworkspace/cli** | Video: **youtu.be/S99_UhOQjNw**

NVIDIA OpenShell — Secure Runtime for AI Agents

Run AI agents safely in an isolated sandbox

ITEM	DETAILS
Core	Sandbox container + YAML policy-based access control
Supported Agents	Claude Code, Codex, Cursor, GitHub Copilot CLI
Security Model	4-layer defense: file / network / process / inference
License	Apache 2.0 (Open Source)

```
openshell sandbox create -- claude    # Run Claude Code in a sandbox
```

> **Infrastructure-level implementation** of Harness Engineering — enforcing security policies outside the agent process
>
> GitHub: **github.com/NVIDIA/OpenShell**

Part II

Tips for Using Claude Code for Free

Getting Started with Claude Code

Installation

```
npm install -g @anthropic-ai/claude-code
```

5 Ways to Use It

ENVIRONMENT	DESCRIPTION
CLI	Run `claude` in the terminal — the most powerful option
VS Code Extension	Use directly in the IDE
JetBrains Extension	IntelliJ, WebStorm, etc.
Desktop App	Dedicated app for Mac / Windows
Web App	claude.ai/code — use right in the browser

> After installation, just type `claude` in the terminal to get started!

Free / Low-Cost Usage Options

METHOD	COST	FEATURES
Ollama + Claude Code	**Free**	Use Claude Code UI with local models
Claude.ai Free Tier	Free	Daily usage limits, basic experience
Anthropic API	Usage-based	Sign-up credits, pay only for what you use
Claude Pro	$20/mo	Recommended for individual developers
Claude Max	$100/mo	Heavy users, near-unlimited usage

> If your company has an AWS/GCP account, you can use Claude Code via **Bedrock / Vertex AI** — billed to your company's cloud budget, no separate Anthropic subscription needed

Where Does Claude Code Run?

Direct Connection (Default)

```
Claude Code CLI  →  Anthropic API  →  Claude model on Anthropic servers
```

Via Company Cloud

```
Claude Code CLI  →  Google Vertex AI API  →  Claude model on GCP servers
Claude Code CLI  →  AWS Bedrock API       →  Claude model on AWS servers
```

- The **model is the same** Claude — only the **server location** differs
- Google is a **major investor** in Anthropic → hosts Claude on Vertex AI
- Use your company's GCP/AWS contract → use Claude **without additional vendor contracts**

Using Claude Code for Free with Ollama

OLLAMA CONNECTS CLAUDE CODE'S TERMINAL UI TO LOCAL MODELS

```
# One line after installing Ollama
ollama launch claude --model kimi-k2.5:cloud
```

ITEM	DETAILS
How It Works	Ollama connects Claude Code UI to local/cloud models
Available Models	Kimi K2.5, Qwen, DeepSeek, Llama, etc.
Cost	**Completely free** when using local models
Advantage	Claude Code's powerful UX + freedom to choose any model

> Official docs: **docs.ollama.com/integrations/claude-code**
>
> Use Claude Code's UI and workflows with zero cost!

CLAUDE.md Memory Hierarchy

Multiple CLAUDE.MD files? → Loaded by priority

PRIORITY	LOCATION	SCOPE
1 (highest)	`/etc/claude-code/CLAUDE.md`	Enterprise/org policy (cannot be overridden)
2	`project/CLAUDE.local.md`	Local override (gitignored, private)
3	`project/CLAUDE.md`	Project level (shared with team, VCS)
4 (lowest)	`~/.claude/CLAUDE.md`	Personal global settings (all projects)

- Enterprise policy **always takes precedence** — individual developers cannot override
- For the rest, **more specific files take priority** (local > project > global)

> Global for personal style, project for team rules, local for your overrides

Andrej Karpathy's CLAUDE.md Guidelines

Andrej Karpathy (ex-Tesla AI / OpenAI) identified core issues with LLM coding:

> **"The model makes wrong assumptions on your behalf and runs with them...**
> **It overcomplicates code... and changes or deletes code it doesn't understand."**

4 Principles (Implemented via CLAUDE.MD)

PRINCIPLE	DESCRIPTION
Think Before Coding	Don't assume — **ask first**, then start
Simplicity First	Implement **only** what was requested, no over-engineering
Surgical Changes	Modify **only what's needed**, no unrelated refactoring
Goal-Driven Execution	Don't dictate actions — provide **success criteria**

Install: `/plugin install andrej-karpathy-skills@karpathy-skills`

GitHub: forrestchang/andrej-karpathy-skills

settings.json — Preventing AI Mishaps Proactively

Configure which commands AI can execute via **allow/deny** in `.claude/settings.json`

ALLOW — Only permitted commands can run

```
"allow": [ "Bash(./mvnw *)", "Bash(git status*)", "Bash(git log*)",
           "Bash(git diff*)", "Bash(git add *)", "Bash(git commit *)" ]
```

DENY — Block dangerous commands entirely

```
"deny": [ "Bash(rm -rf *)", "Bash(git commit --no-verify*)" ]
```

- `rm -rf` → **Block file deletion**
- `--no-verify` → **Block Harness bypass** (most important!)

> settings.json

4 Tips for Efficient Usage

1. Use `CLAUDE.MD`

Place a rules file at the project root → AI **reads it automatically** → no need to repeat the same instructions

2. Compress Context with `/COMPACT`

When conversations get long, use `/compact` → summarizes previous content → **saves tokens**

3. Separate Sessions

Run Design and Execute in **separate sessions** → prevents context window exhaustion

4. Leverage SKILLS & PLUGINS

Automate repetitive tasks with Skills → **no need for the same prompts** every time

> Following just these 4 tips can **save 50%+ in costs**

Part III

Agentic Development with Harness

github.com/tedwon/agentic-dev-playbook-with-harness

What Is the Agentic Development Playbook?

A systematic process where AI agents and humans **divide roles** to develop software

PHASE 1

Design

Human: Define requirements

Agent: Research, design proposals

→

PHASE 2

Execute

Human: Review, adjust direction

Agent: Code, tests

→

PHASE 3

Code Review

Human: Review code

Agent: Run CI

→

PHASE 4

Validate

Human: Final approval

Agent: Security scans

> **Humans steer.** Humans set the direction — **Agents execute.** AI carries it out.

What Is Agentic AI? — The Academic Definition

> "A system that autonomously perceives context, plans, and executes multi-step actions to achieve goals"
> — arxiv.org/pdf/2603.27075

Components of a True AI Agent

COMPONENT	DESCRIPTION
Goal-driven	Receives a goal and executes autonomously
Planning	Breaks tasks into steps
Tool Use	Calls APIs, code, external systems
Memory	Maintains state across steps
Loop (ReAct)	Think → Act → Observe → Repeat
Multi-agent	Multiple agents collaborate (optional)

Sources: arxiv.org/html/2601.02749 · arxiv.org/pdf/2601.12560 · arxiv.org/pdf/2510.25445

The Truth About "Agentic" — Let's Be Honest

> Is this a **true AI agent system**? Or is it a **structured framework for systematically leveraging AI**?

- It's NOT a fully autonomous multi-agent system
- It IS a **structured workflow designed to behave agent-like**
- Most real-world "agentic" systems: one LLM + sequential calls + prompt chaining

> **"Agentic is frequently used as a marketing term"** — arxiv.org/pdf/2506.01463
> Not real agents talking to each other, but **orchestrated LLM workflows**

Agentic Maturity — Where Are We?

LEVEL	DESCRIPTION	EXAMPLE
0	Single LLM response	Ask one question to ChatGPT
1	Prompt engineering	Systematic prompt design
2~3	**Structured workflow + tool use**	**← Our Playbook**
4	True multi-agent system	AutoGen, LangGraph, CrewAI

> **Agentic ≠ Multiple AIs collaborating autonomously**
>
> **Reality: Agentic = LLM + Loop + Structured prompts + Tool use**
>
> Yet even this is **creating substantial value in practice**.

Benefits of the Playbook

What happens when you tell AI "just build it" without structure?

- AI **dumps code without direction** → unwanted results
- The "AI decides on its own" problem persists

What the Playbook Solves

- **Design → Execute separation**: humans set the direction first
- **Human-in-the-loop**: humans review and approve at each step
- **Spec + Plan documentation**: agree on what AI will build beforehand (enables handoff to new sessions)

> Thanks to the Playbook, we can prevent AI from **running off in the wrong direction**.

But the Playbook Alone Wasn't Enough

- AI confidently commits **code that doesn't compile**
- Basic mistakes like **System.out.println**, hardcoded passwords
- Implements features **without test files**
- Even attempts to **modify rule files themselves** to bypass verification

> The Playbook defines **"what to do"**
> but there was no structure to **automatically verify "whether it was done correctly"**.

That's Why We Adopted Harness Engineering

Playbook + Harness = A Satisfying Level of Quality

	PLAYBOOK ONLY	PLAYBOOK + HARNESS
Direction setting	**O**	**O**
Automated verification	X	**7 automated checks**
Auto-correction of mistakes	X	**Self-Correction Loop**
Rule file protection	X	**File Protection Hook**
Immediate feedback	X	**Post-edit compile check**

> If the Playbook is the **rudder**, then Harness is the **guardrail**.
> Together, they finally reach a level where we can **trust and delegate to the AI agent**.

The Birth of Harness Engineering

Feb–Mar 2026: The term "Harness" emerged and spread across the industry

DATE	AUTHOR	KEY CONTRIBUTION
2026. 2. 5	**Mitchell Hashimoto**	First proposed the "Engineer the Harness" concept
2026. 2. 11	**OpenAI (Codex team)**	Formalized `Agent = Model + Harness`
2026. 3. 24	**Anthropic**	Deep dive into Harness Design — 3-Agent architecture
2026. 4	**This project**	Applied Harness Engineering in practice — JBUG Seminar

> In just 3 months, an idea from a personal blog → spread to OpenAI/Anthropic's official engineering methodology
> And we **applied it directly to a Quarkus project**

Mitchell Hashimoto — "Engineer the Harness" (2026. 2. 5)

The HashiCorp founder shared his **6-stage AI adoption journey**

PHASE	STAGE	DESCRIPTION
1	Drop the Chatbot	Move beyond chatbots to **agents**
2	Reproduce Your Own Work	Repeat the same task manually + with AI to learn
3	End-of-Day Agents	Delegate research to agents when leaving work
4	Outsource the Slam Dunks	Delegate straightforward tasks to agents
5	**Engineer the Harness**	**Structurally prevent agent mistakes**
6	Always Have an Agent Running	Keep agents running at all times

> **"Every time an agent makes a mistake, build a structure so that mistake never happens again."**

- **AGENTS.md**: **"Every line in that file is based on bad agent behavior"**

> Source: mitchellh.com/writing/my-ai-adoption-journey

OpenAI — Agent = Model + Harness (2026. 2. 11)

- **A small team (3→7 engineers), 0 lines of code written manually, 1,500 PRs, 1M lines** — all written by agents
- Presented the `Agent = Model + Harness` formula
- LangChain: Improved only the Harness → **Terminal Bench 2.0** from 30th → 5th place (no model change)

> **"The hard part isn't the Agent — it's the Harness."** — OpenAI

ANTHROPIC — DEEP DIVE INTO HARNESS DESIGN (2026. 3. 24)

- Multi-agent harness: **Initializer (task decomposition) → Coding Agent (implementation) → verification loop**
- Solo agent ($9, doesn't work) vs Harness ($200, works perfectly) → **structure, not cost, determines quality**

> **"Every component of the harness encodes an assumption about what the model can't do on its own."**

What We Applied to Our Project

SOURCE	OUR IMPLEMENTATION
Hashimoto's AGENTS.md	`CLAUDE.md` + `AGENTS.md` + `CHECKLIST.md` *
OpenAI's Feedforward/Feedback	Pre-commit Hook (7 checks) + File Protection
Anthropic's session separation	Phase 1 (Design) and Phase 2 (Execute) in separate sessions
Anthropic's Evaluator	Reviewer sub-agent + Human Review

* `CHECKLIST.md` is **our original idea** not found in external references — declaratively documenting verification rules

> Turning blog post **theory** → into **code** in a real project
>
> GitHub: **github.com/tedwon/agentic-dev-playbook-with-harness**

The Reality of AI Coding

80% of developers use AI coding tools, but... — Stack Overflow 2025

- **40% of AI-generated code contains security vulnerabilities** — Pearce et al. 2022
- Only **29% of developers trust** AI code in production — Stack Overflow 2025
- Increasing **incident reports** from teams merging AI code without review

The Root Cause

> It's not that AI models lack capability.
> **The absence of verification structure (Harness) is the problem.**

Agent = Model + Harness

	MODEL (ENGINE)	HARNESS (CONTROL SYSTEM)
Role	Code generation, reasoning, analysis	Verification, correction, guardrails
Analogy	700 HP engine	Steering wheel + brakes + dashboard
Alone	Fast but **dangerous**	Slow but **safe**
Combined	**Fast and safe**	

> **Harness design is as important as model selection.**
>
> Good model + bad Harness = dangerous speed.
>
> Average model + good Harness = reliable results.

What Is Harness Engineering?

Designing structures that allow AI agents to **work autonomously yet safely**

Two Types of Control

CONTROL TYPE	WHEN?	HOW?	IMPLEMENTATION FILES
Feedforward (Prevention)	**Before** mistakes	Make agents read rules in advance	`CLAUDE.md` , `AGENTS.md`
Feedback (Correction)	**After** mistakes	Detect errors → guide fixes	Pre-commit hooks

> **Core Principle:**
> The agent reads the rules (Feedforward), self-corrects on violations (Feedback), and iterates **without human intervention**.

Self-Correction Loop

The core mechanism where agents **self-correct without human intervention**

```
Agent writes code
    |
Attempts git commit
    |
Pre-commit Harness ──→ Runs 7 verification checks
    |-- All pass? ──→ Commit succeeds
    |-- Failure? ──→ Commit blocked + detailed error messages
                  |
         Agent reads errors and fixes them
                  |
         Retries commit (repeat)
```

> For this loop to work, error messages must be **in a format the LLM can understand**.

LLM-Optimized Error Messages

Harness error messages are designed **for AI agents**, not humans

```
[FAIL] QUAL-01 | System.out.println found in QuoteResource.java

  WHAT:  System.out.println("Loading quotes...");
  WHY:   Production code must use structured logging.
  FIX:   Replace with org.jboss.logging.Logger

  EXAMPLE:
    private static final Logger LOG = Logger.getLogger(QuoteResource.class);
    LOG.info("Loading quotes...");
```

Error Message Components

Rule ID → What went wrong → Why it's wrong → How to fix it → Code example

7 Automated Verification Rules

7 checks run simultaneously before every `git commit`

ID	CHECK	DESCRIPTION
BUILD-01	Compile verification	`./mvnw compile -q`
BUILD-02	Test verification	`./mvnw test`
BUILD-03	Formatting verification	`./mvnw spotless:check -q`
QUAL-01	No System.out	Enforce `org.jboss.logging.Logger` usage
QUAL-02	No secrets	Enforce `@ConfigProperty` usage
CONV-01	Conventional Commits	`feat(scope): subject` format
CONV-02	Test coverage	`*Test.java` required for every `@Path` class

> **Design principle:** Show all failures **at once** so the agent can fix them **all at once**.

3-Hook System

HOOK	WHEN	ACTION	ON FAILURE
Pre-commit Harness	On `git commit`	Runs 7 verification checks	Commit **blocked**
File Protection	On file modification	Detects rule file tampering	Modification **blocked**
Post-edit Verify	After `.java` edits	Immediate compile check	Warning only

> **Why File Protection is needed:**
> The **easiest way** for an agent to "fix" verification failures is to modify the rules themselves. This **blocks that entirely**.

Demo Project: Quote of the Day API

This entire project was developed using the Agentic Development Playbook.

REST API ENDPOINTS

```
GET /api/quotes            Full list of quotes (?category=programming filter)
GET /api/quotes/random     One random quote
GET /api/quotes/{id}       Lookup by ID (404 if not found)
```

TECH STACK

Java 21 + Quarkus 3.34 + Spotless + REST Assured + JUnit 5 (11 tests)

GITHUB

github.com/tedwon/agentic-dev-playbook-with-harness

> Playbook, hook scripts, and demo recordings are all public

Demo: Writing Violation Code

Intentionally writing code that **violates 4 rules**

```
public class TimeResource {
    @GET
    public String getTime() {
        System.out.println("time requested");     // QUAL-01 violation
          return LocalTime.now().toString();       // BUILD-03 violation (indentation)
    }
}
// No test file                                   // CONV-02 violation
// git commit -m "added stuff"                    // CONV-01 violation
```

Demo: Harness Blocks and Corrects

```
[FAIL] QUAL-01  : System.out.println found → Use Logger
[FAIL] BUILD-03 : Formatting violation → Run spotless:apply
[FAIL] CONV-02  : No test for TimeResource → Create TimeResourceTest.java
[FAIL] CONV-01  : Bad commit msg → Use feat(quotes): add time endpoint

RESULT: 3/7 passed, 4/7 FAILED — commit blocked
```

AGENT'S AUTO-CORRECTION (No Human Intervention)

Agent reads all 4 violations **at once**, fixes them **at once** → retries → **7/7 passed**

Live Demo: Issue #2 — Quote AI Chatbot

Applied the Agentic Development Playbook's 4 phases **in practice** to implement an AI chatbot feature

PHASE	DESCRIPTION	ARTIFACTS
Design	Brainstorm → Spec → Plan	Spec · Plan
Execute	Implement 7 tasks → Harness 7/7 passed	6 new files + 3 modified
Review	Create PR → Code review	PR #3
Validate	SpotBugs + SBOM + local testing	13 tests passed (base 11 + 2 new)

> Issue: #2 · PR: #3 · Walkthrough: Walkthrough

Demo Recordings (asciinema)

DEMO	DESCRIPTION	LINK
Self-Correction	Violation code → harness blocks → auto-fix → 7/7 passed	Play
Build & Test	13 tests passed + harness 7/7 checks	Play
Live API	Ollama (qwen3:1.7b) real-time AI responses	Play
Security	SpotBugs static analysis + CycloneDX SBOM generation	Play

> All recordings: asciinema.org/~tedwon
>
> Docs: github.com/.../docs

3 Key Risks

RISK	DESCRIPTION	MITIGATION
Skill Atrophy	Core competencies erode through AI dependency	**Don't approve code you can't explain**
Uncontrolled Autonomy	"Just build it" without design → wrong results	Set direction first with **Playbook**
Blind Trust	AI can be confidently wrong	**Always verify** with domain expertise

> AI is a tool that **amplifies** experts, not **replaces** them.
>
> **"If you can't explain the code the agent wrote, don't approve it."**

Project Structure

```
agentic-dev-playbook-with-harness/
+-- CLAUDE.md / AGENTS.md / CHECKLIST.md  ⟵ Feedforward rules
+-- .claude/
|   +-- settings.json                     ⟵ Hook configuration
|   +-- hooks/
|       +-- pre-commit-harness.sh         ⟵ 7 verification checks (Feedback)
|       +-- protect-files.sh              ⟵ File protection
|       +-- post-edit-verify.sh           ⟵ Immediate compile check
+-- src/main/java/                        ⟵ Quarkus REST API
+-- src/test/java/                        ⟵ 11 tests
+-- docs/                                 ⟵ ADR, Spec, Plan storage
+-- demo/                                 ⟵ Demo recordings, violation examples
```

> Demo branch: **github.com/tedwon/agentic-dev-playbook-with-harness/tree/feat/issue-2-quote-ai-chatbot**

Key Messages

Three Things to Remember

Today's Key Takeaways

TOOL	ROLE
MCP AI Assistant	Unified work tools — diverse services, 120+ tools
Skills & Agents	Extended specialist capabilities — 33 skills, 4 agents
Claude Code	The core engine for development — CLI, IDE, Web
Harness Engineering	Structure for AI to **execute safely and autonomously**
OpenClaw	Access AI anytime, anywhere

1. HARNESS DESIGN IS AS IMPORTANT AS MODEL SELECTION

2. START WITH WORKFLOWS, DEPLOY AGENTS ONLY WHERE NEEDED

3. YOU MUST UNDERSTAND THE CODE YOU APPROVE

Getting Started Today

STEP 1: INSTALL CLAUDE CODE + WRITE `CLAUDE.MD`

Tell the agent your project rules, coding conventions, and prohibited actions

STEP 2: ADD ONE PRE-COMMIT HOOK

Start with at least **compile + test** automated verification

STEP 3: DEVELOP YOUR FIRST FEATURE WITH THE 4-PHASE PLAYBOOK

Run through one cycle of Design → Execute → Review → Validate

> GitHub: **github.com/tedwon/agentic-dev-playbook-with-harness**
> Playbook, hook scripts, and demo code are all public

Thank You

Q&A

Ted Jongseok Won

JBUG Korea

**"Your software engineering expertise isn't disappearing —
it's becoming more powerful when combined with AI."**

"Humans steer. Agents execute."

AI-Driven Software Engineering

References

Harness Engineering · Mitchell Hashimoto — My AI Adoption Journey · OpenAI — Harness Engineering · Anthropic — Harness Design for Long-Running Apps · Awesome Harness Engineering

Agentic Development · Anthropic — Building Effective Agents · Anthropic — 2026 Agentic Coding Trends · Karpathy CLAUDE.md Guidelines

Papers · Agentic AI Definition — arxiv 2603.27075 · Agentic AI Survey — arxiv 2601.02749 · Agentic as Marketing Term — arxiv 2506.01463

Tools · Claude Code · MCP · Ollama + Claude Code · Google Workspace CLI · NVIDIA OpenShell · Quarkus AI

This Project · GitHub Repo · Demo Branch · PR #3 · Demo Recordings · Walkthrough

Inspiration · Julia Evans — Write a Brag Document · Pearce et al. — AI Code Security · Stack Overflow 2025 Survey